

University Cafeteria Ordering System

Phases 1 – 4 · Full Technical Report

Requirement Discovery · Gherkin Scenarios · API Contracts
Test-Driven Implementation · QA & Testing Report

CampusBite Platform · Software Engineering Course · 2025

Phase 1 — Requirement Discovery & Traceability

1. Actor Classifications

1.1 Primary Actors

Primary actors directly initiate system use cases.

Student / User

The central human actor. Initiates every customer-facing flow: authentication, menu browsing, cart management, order placement, payment, tracking, and feedback.

Traced to: FR01–FR08, FR09–FR17, FR20–FR26, FR33–FR38, FR47

Rationale: All UI design (NFR25–NFR27) is calibrated to their mobile-first usage patterns (NFR26). They are the primary beneficiary of every functional requirement.

Staff

Operational actor who fulfils orders. Advances order state through the state machine (CONFIRMED → PREPARING → READY → COLLECTED). Has unique permissions not shared with Student or Admin.

Traced to: FR35, FR39, FR41, FR45, FR51

Rationale: Has unique permissions not shared with Student or Admin. Is an explicit actor in every state machine transition from CONFIRMED onward.

Administrator

Highest-privilege human actor. Manages users, menu, system configuration, financial balances, refund queues, and flagged order review.

Traced to: FR07, FR18–FR19, FR24, FR39, FR49–FR56

Rationale: Explicitly declared in FR07 with a named role. Has non-overlapping permissions from Staff (NFR14).

1.2 Supporting Actors

Supporting actors provide services consumed by the system.

Payment Gateway

PCI-DSS-compliant external payment processor. Handles all card processing asynchronously via webhooks. Provides double-charge prevention via idempotency keys.

Traced to: FR27–FR32, FR42, FR44, NFR03, NFR08, NFR17

Rationale: The most dependency-intensive external actor. Has its own circuit-breaker pattern (NFR08) and is the sole trigger for PAYMENT_PENDING → CONFIRMED transitions.

Notification Service

Async background job responsible for sending in-app and email order confirmation notifications (FR33). Decoupled from order processing to avoid blocking the main flow.

Traced to: FR33, NFR28
Rationale: Explicitly described as an async background job in FR33. Its decoupled nature confirms it as a supporting rather than primary actor.

1.3 Offstage Actors

Offstage actors are invisible stakeholders who shape requirements.

University Finance / Auditors

Never interact with the system directly but drive immutable audit log requirements (NFR20, NFR32), financial reporting, and 3-year data retention policy (NFR31).

Traced to: FR46, NFR20, NFR31, NFR32

Malicious / Erroneous User

An implicit threat actor whose existence is acknowledged in SRS §2 ("resilient to malicious or erroneous input"). Every security requirement is fundamentally a response to this actor.

Traced to: FR03, FR14–FR16, FR24, NFR16, NFR19, NFR21

2. Feature-to-Requirement Traceability Heatmap

Each row maps a system feature to its source FR IDs, Gherkin scenario coverage, and state machine alignment. Coverage symbols: ✓ Full ~ Partial ✗ None.

Authentication			
Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Login with university credentials	FR01, FR02	✓ Full	N/A
Account lockout (5 failed attempts)	FR03	✓ Full	N/A
Session expiry on inactivity	FR04	✓ Full	N/A
Logout & session invalidation	FR05	~ Partial	N/A
Password reset (time-limited link)	FR06	~ Partial	N/A
Role assignment by admin	FR07	✗ None	N/A
Reject suspended/expired accounts	FR08	✓ Full	N/A

Menu & Cart

Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Browse menu by category	FR09, FR10	✓ Full	N/A
Search menu by keyword	FR10	✓ Full	N/A
Add items to cart; out-of-stock blocked	FR11, FR12	~ Partial	N/A
Voucher application & validation	FR13–FR16	✓ Full	N/A
Cart lock at checkout	FR17	~ Partial	DRAFT→PLACED
Menu item management (Admin)	FR18, FR19, FR52	✗ None	N/A

Order Placement

Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Place order with unique ID (idempotent)	FR20, FR23	~ Partial	DRAFT→PLACED
Real-time stock check at placement	FR21, FR22	~ Partial	N/A
Bulk/suspicious order detection	FR24, FR25	✗ None	N/A

Payment

Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Payment method selection	FR26	~ Partial	PLACED→PAYMENT_PENDING
Online gateway payment	FR27, FR30	~ Partial	PAYMENT_PENDING→CONFIRMED
Gateway failure & retry	FR28, FR29	~ Partial	PAYMENT_PENDING→PAYMENT_FAILED
Meal Plan payment & balance check	FR31	✗ None	N/A
Wallet atomic deduction	FR32	✗ None	N/A
Order confirmation notification	FR33	✗ None	N/A

Order Lifecycle

Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Order state machine	FR34, FR35	~ Partial	✓ Full
Real-time order tracking (SSE)	FR36	✗ None	N/A
User cancellation (2-min window)	FR37, FR38	~ Partial	PLACED→CANCELLED
Admin/Staff forced cancellation	FR39	✗ None	CONFIRMED→CANCELLED
Auto-cancel abandoned checkout	FR40	✗ None	PAYMENT_PENDING→CANCELLED
Stock inconsistency detection	FR41	✗ None	N/A

Refunds / Feedback / Admin

Feature / Use Case	FR IDs	Gherkin Coverage	State Machine
Automatic refund on cancellation	FR42, FR43	✗ None	N/A
Refund gateway failure handling	FR44	✗ None	N/A
Partial refund on partial fulfilment	FR45	✗ None	N/A
Immutable refund audit log	FR46	✗ None	N/A
Post-order rating & feedback	FR47	✗ None	→COMPLETED
Average rating on menu page	FR48	✗ None	N/A
Admin feedback moderation	FR49	✗ None	N/A
User account management	FR50, FR51	✗ None	N/A
Reports & exports	FR53	✗ None	N/A
System config management	FR54	✗ None	N/A
Wallet/Meal Plan top-up (step-up auth)	FR55	✗ None	N/A
Flagged order review & approval	FR56	✗ None	N/A

3. Non-Functional Requirement Coverage

NFRs that are not fully traced to Gherkin scenarios or test strategies are listed below. Gaps represent test debt and must be prioritised in the next sprint.

ID	Category	Requirement Summary	Coverage	Gap / Note
NFR06	Availability	99.5% uptime during cafeteria hours	~ Partial	No SLA test specified
NFR07	Availability	Maintenance windows off-hours	✗ None	No graceful-degradation spec
NFR08	Availability	Payment circuit breaker	✓ Full	Traced to FR28/FR29
NFR09	Scalability	Horizontal scale to 3× peak	~ Partial	Architecture referenced; no load test
NFR10	Scalability	10k orders/day without degradation	~ Partial	Indexes specified; load test missing
NFR11	Scalability	500 concurrent lock acquisitions/s	~ Partial	Redis/DB-level; no bench
NFR12	Security	TLS 1.2+ for all data in transit	✓ Full	Standard infra requirement
NFR13	Security	bcrypt cost ≥12; no plaintext passwords	✓ Full	Explicit in SRS
NFR16	Security	Parameterised SQL / ORM only	✓ Full	CI static analysis gate
NFR21	Security	OWASP Top 10 mitigations	~ Partial	No explicit test scenarios
NFR25	Usability	Plain-language error messages	~ Partial	Catalogue not yet produced
NFR26	Usability	Responsive (min 375px) all browsers	✗ None	No test or spec
NFR27	Usability	WCAG 2.1 Level AA	✗ None	HR-03 gap — ARIA live regions missing
NFR29	Observability	/health endpoint with deep checks	~ Partial	Described; no test
NFR30	Observability	Alerting within 2 min on error/latency	✗ None	No alerting spec written
NFR31	Compliance	3-year personal data retention + purge	✗ None	No purge job designed

4. Orphaned Hidden Requirements (Persona Discovery)

The following hidden requirements were identified in Phase 1 persona analysis. None are implemented in the current SRS and represent risk to delivery.

ID	Hidden Requirement	Nearest FR	Status	Risk
HR-01	Guest / Visitor Payment Mode	—	✗ None	Non-university visitors cannot pay
HR-02	Meal Plan Activation Lag	FR31 (partial)	~ Partial	New student Day-One edge case
HR-03	ARIA Live Regions for SSE Updates	NFR27 (gap)	✗ None	Accessibility — screen reader users
HR-04	Staff Order Assignment / Idempotent Transitions	FR35 (gap)	~ Partial	Two-staff simultaneous handover
HR-05	IdP Outage Degraded Mode	NFR06 (gap)	✗ None	Full system lockout at peak lunch
HR-06	Reorder from History (deactivated items)	FR36 (gap)	✗ None	Inconsistent UX for returning users
HR-07	Cross-Device Cart Invalidation	FR12, FR17 (gap)	~ Partial	Stale cart causes silent oversell

5. Coverage Summary

Dimension	✓ Full	~ Partial	✗ None / Gap
Gherkin Scenario Coverage (39 features)	8 (21%)	11 (28%)	20 (51%)
Edge Case Test Coverage (24 edge cases)	10 (42%)	8 (33%)	6 (25%)
NFR Explicit Tracing (32 NFRs)	7 (22%)	9 (28%)	16 (50%)
Hidden Requirements Addressed (7 HRs)	0 (0%)	3 (43%)	4 (57%)

Priority Actions

- Write Gherkin for FR07, FR24–FR25, FR31–FR33, FR36, FR39–FR56 — representing 51% of untested features.
- Add WCAG 2.1 / ARIA test suite (HR-03, NFR27).
- Design IdP degraded-mode circuit breaker (HR-05, NFR06).
- Specify and test NFR26 responsive breakpoints.
- Create purge job design document for NFR31 (3-year retention).

6. Persona Discovery — Hidden Requirements

HR-01 · The Graduation-Day Visitor

A parent visits campus on graduation day. They have no university account, want to buy lunch from the cafeteria. The SRS defines only four payment methods (FR26): Online, Cash on collection, Wallet, and Meal Plan. Three of the four require a university account. Cash on collection is the only viable path, but it is not surfaced without login.

Hidden Requirement Identified

Guest / Visitor Payment Mode — the system has no guest checkout or visitor payment flow.

HR-02 · The New Student on Day One

A student who enrolled this morning. Their university account exists in the directory, but their Meal Plan has not been activated and their Wallet balance is zero. FR31 rejects Meal Plan payment if insufficient balance, but makes no distinction between 'not activated' and 'zero balance' — two states that require different resolution paths.

Hidden Requirement Identified

Meal Plan Activation Lag & Zero-Balance Wallet — two distinct error states must be surfaced separately.

HR-03 · The Accessibility-Dependent Student

A visually impaired student who uses a screen reader on their iPhone. They rely entirely on keyboard navigation and ARIA labels. NFR27 mandates WCAG 2.1 Level AA. FR36 pushes real-time order updates via SSE. There is no specification for ARIA live regions to announce SSE status changes to assistive technology.

Hidden Requirement Identified

Real-Time Order Status Accessibility — SSE updates require ARIA live region annotations.

HR-04 · The Shift-Change Staff Member

A staff member's shift ends mid-order. They hand off to a colleague. The incoming staff member logs in and navigates to the same order simultaneously. The SRS defines Staff as a role but never specifies whether orders are assigned to specific staff members or whether two staff can advance the same order concurrently.

Hidden Requirement Identified

Staff Session Handover & Order Assignment — idempotent state transitions required for concurrent staff access.

HR-05 · The IdP Outage User

At 12:15 PM (peak lunch), the university's Active Directory has a 10-minute outage. Hundreds of students simultaneously cannot log in. NFR08 isolates payment gateway failures via a circuit breaker, but no equivalent resilience pattern is specified for authentication provider failures.

Hidden Requirement Identified

IdP Unavailability Degraded Mode — no circuit breaker or cached-session fallback is specified.

HR-06 · The Chronic Re-Orderer

A student places the same order every day for a month using the order history. One day, a menu item they always order is deactivated by admin (FR52). FR36 specifies order history shows all past orders with final status, but no requirement defines a reorder action or what happens when a reorder includes a deactivated item.

Hidden Requirement Identified

Reorder Flow from Order History — no requirement covers reorder with deactivated/changed items.

HR-07 · The Multi-Device Power User

A student opens the app on their phone, builds a cart, then opens it on their laptop. They check out on the laptop. The phone still shows the cart as editable. FR12 states 'Cart persists for session duration.' FR17 locks the cart at checkout. FR23 prevents duplicate orders. But no requirement addresses cross-device cart state synchronisation.

Hidden Requirement Identified

Cross-Device Cart Synchronization & Stale Cart Invalidation — second device must receive CART_INVALIDATED.

Phase 2 — Gherkin Scripts, QA Refinement & API Contracts

Part A: Gherkin Scenarios

FR01 — Login with University Credentials

Feature: University Cafeteria – Authentication

Scenario: Successful login with valid credentials

Given a registered student with email "ali@university.edu" and a valid password
When the student submits the login form with correct credentials
Then the system authenticates against the institutional IdP
And a JWT access token (TTL 30 min) and refresh token (TTL 7 days) are issued
And the student is redirected to the menu home page

Scenario: Login rejected for suspended account

Given a student account that has been suspended by the administrator
When the student attempts to log in
Then the system returns HTTP 401
And displays: "Your account has been suspended. Contact the university helpdesk."
And no token is issued

Scenario Outline: Login rejected for invalid credentials

Given a login attempt with email <email> and password <password>
When the form is submitted
Then the system returns HTTP 401
And the response body contains "Invalid credentials"

email	password	reason
unknown@ext.com	anyPass1!	Unregistered email
ali@university.edu	wrongPass!	Wrong password
expired@university.edu	validPass1!	Expired account

FR03 — Account Lockout After Failed Attempts

Scenario: Account locked after 5 consecutive failures

Given a valid student account "ali@university.edu"
And 4 failed login attempts have already been recorded
When the student submits a 5th incorrect password
Then the account is locked for exactly 15 minutes
And the system logs the lockout event with timestamp and originating IP address
And the response returns HTTP 403: "Account locked. Try again in 15 minutes."

Scenario: Login succeeds after logout period expires

Given the account was locked 15 minutes ago
When the student submits the correct password
Then the system unlocks the account
And issues a valid JWT token
And the failed-attempt counter is reset to zero

FR04 — Session Expiry on Inactivity

Scenario: Session invalidated after 30 minutes of inactivity

Given a student is logged in with an active JWT token
And the student has not made any API request for 30 minutes
When the student sends any authenticated request
Then the server returns HTTP 401: "Session expired. Please log in again."
And the JWT token is removed from the Redis token store

FR09–FR11 — Browse and Search Menu

Scenario Outline: Filter menu by category

Given the menu contains items in categories: Meals, Beverages, and Snacks
When the student requests the menu with filter <category>
Then only items belonging to <category> are returned
And each item shows: name, price in EGP, description, image URL, and stock status

category
Meals
Beverages
Snacks

Scenario: Search returns results within 1 second

Given the menu contains 50 active items
When the student searches for keyword "koshary"
Then matching items are returned within 1,000 ms at the 95th percentile
And out-of-stock items display an "Unavailable" badge and cannot be added to cart

Scenario: Empty search result shows friendly message

Given no menu item matches the keyword "xyz_nonexistent"
When the student performs the search
Then the response body contains: "No items found for 'xyz_nonexistent'. Try a different keyword."
And the HTTP status is 200 (not 404)

FR13–FR16 — Voucher Application

Scenario: Valid voucher applied successfully

Given a student has a cart total of 120 EGP
And voucher code "SAVE20" offers 20% discount, has not been used, and is not expired
When the student applies the voucher code
Then the cart total is updated to 96 EGP
And the voucher is marked as used for this student atomically

Scenario Outline: Invalid voucher rejected with specific reason

code	cart_total	error message
USED01	80 EGP	Voucher has already been used by your account.
EXPD01	80 EGP	Voucher has expired.
MIN100	50 EGP	Minimum order of 100 EGP required for this voucher.
RVKD01	80 EGP	Voucher is no longer valid.

Scenario: Voucher over-discount floors cart total to zero

Given a student has a cart total of 30 EGP
And voucher "BIGDISCOUNT" offers a 100 EGP flat discount
When the student applies the voucher
Then the cart total is set to 0 EGP – no negative amount is stored or transmitted

Scenario: Concurrent voucher use — first request wins

Given student "ali" is logged in on Device A and Device B simultaneously
And voucher "ONCE01" is single-use
When both devices submit the voucher at the same millisecond
Then exactly one request succeeds and the other returns HTTP 409: "Voucher already applied."
And the voucher is consumed exactly once in the database

FR20–FR25 — Order Placement**Scenario: Successful order placement assigns UUID and timestamp**

Given a student has a locked cart with 2 available items
When the student submits the order
Then the system creates an order with a UUID v4 order ID
And the order timestamp is recorded to millisecond precision in UTC
And the order status is set to PLACED
And a pessimistic stock lock is acquired on all ordered items

Scenario: Duplicate order rejected within 60-second window

Given a student placed order "ORD-001" 30 seconds ago
And the cart fingerprint has not changed
When the student re-submits the same cart
Then the system returns the existing order "ORD-001" (HTTP 200)
And no new order record is created

Scenario: Suspicious order routed to admin review queue

Given the suspicious-order threshold is set to total > 500 EGP
And a student submits an order totalling 750 EGP
When the order is validated
Then the order status is set to FLAGGED
And the order appears in the admin review queue
And the student receives: "Your order is under review. You will be notified within 60 minutes."

FR26–FR32 — Payment Processing

Scenario: Online payment succeeds via gateway webhook

Given a student has a CONFIRMED cart and selects online card payment
When the payment gateway sends a success webhook within 10 seconds
Then the order status transitions from PAYMENT_PENDING to CONFIRMED
And the stock lock is converted to a committed decrement
And an order confirmation notification is dispatched within 30 seconds

Scenario: Gateway timeout releases stock lock

Given a student is in PAYMENT_PENDING status
When no webhook is received within 10 seconds
Then the order status is set to PAYMENT_FAILED
And all stock locks for this order are immediately released
And the student sees: "Payment timed out. Your cart is available to retry."

Scenario: Wallet deduction — insufficient balance rejected

Given a student wallet balance is 45 EGP
And the order total is 60 EGP
When the student selects Wallet as the payment method
Then the system returns HTTP 402: "Insufficient wallet balance. Current: 45 EGP. Required: 60 EGP."
And the order remains in PAYMENT_PENDING
And no deduction is made to the wallet

FR34–FR41 — Order Lifecycle & Cancellation

Scenario Outline: Valid forward state transitions

from_state	to_state	actor	trigger
PLACED	PAYMENT_PENDING	User	Payment method selected
PAYMENT_PENDING	CONFIRMED	System	Gateway success webhook
CONFIRMED	PREPARING	Staff	Staff advances status
PREPARING	READY	Staff	Staff marks ready
READY	COLLECTED	Staff	Staff confirms collection
COLLECTED	COMPLETED	System	Auto-complete after 2 hours

Scenario: User cancels order within 2-minute window

Given an order is in PLACED status and was created 90 seconds ago
When the student requests cancellation
Then the order status is set to CANCELLED
And a refund is initiated if payment was made
And the cancellation is logged with actor = User and reason = Customer request

Scenario: PAYMENT_PENDING order auto-cancelled after 10 minutes

Given an order has been in PAYMENT_PENDING status for 10 minutes
When the scheduled cleanup job runs
Then the order status is set to CANCELLED
And all associated stock locks are released
And the student is notified: "Your pending order was automatically cancelled due to payment timeout."

FR47–FR49 — Feedback & Ratings

Scenario: Student submits rating for completed order

Given an order has status COMPLETED
And the student has not yet rated this order
When the student submits a 4-star rating with comment "Good portion size!"
Then the rating is stored linked to the order and student
And the average rating for the ordered item is recalculated within 5 minutes
And the rating cannot be edited after submission

Part B: QA Refinement — Vague NFR Audit

Every vague adjective from Phase 1 NFRs has been replaced with a measurable technical metric. The table below constitutes the official senior QA audit log.

ID	Category	Vague Term	Refined Measurable Metric
NFR01	Performance	"quickly"	Menu page (full item list) loads ≤ 500 ms at p95
NFR02	Performance	"fast"	Order placement API responds ≤ 300 ms at p95
NFR03	Performance	"promptly"	Payment webhook consumed and processed ≤ 2 s
NFR04	Performance	"fast"	Full-text search API responds ≤ 1,000 ms at p95
NFR05	Performance	"reasonably fast"	Admin report generation ≤ 5 s for ≤ 90-day range
NFR06	Availability	"highly available"	System availability ≥ 99.5% during cafeteria hours

ID	Category	Vague Term	Refined Measurable Metric
NFR09	Scalability	"scale well"	API tier must horizontally scale to 3× peak load
NFR11	Scalability	"efficient"	Pessimistic stock lock: 500 acquisitions/s sustained
NFR12	Security	"securely"	All data in transit must use TLS 1.2+
NFR13	Security	"safely"	Passwords stored using bcrypt with cost factor ≥ 12
NFR14	Security	"secure"	All API endpoints protected with JWT Bearer auth
NFR19	Security	"prevent abuse"	Unauthenticated endpoints: 20 req/min/IP rate limit
NFR22	Data Integrity	"inconsistent state"	Stock decrement, payment debit, order update in one ACID transaction
NFR25	Usability	"friendly"	Every user-facing error includes plain-language description
NFR26	Usability	"mobile-friendly"	All pages responsive at min viewport width 375 px
NFR30	Observability	"quickly / things go wrong"	Alerts sent to on-call team within 2 minutes of threshold breach

Part C: UML Diagrams

The following diagrams provide the full system interaction and behaviour models for the University Cafeteria Ordering System, covering all nine major functional areas.

System Sequence Diagrams (SSD)

Nine SSDs illustrate the message exchanges between actors and system components for each major use case: Authentication, Menu & Cart, Voucher, Order Placement, Payment, Order Lifecycle, Refund, Feedback, and Admin Operations.

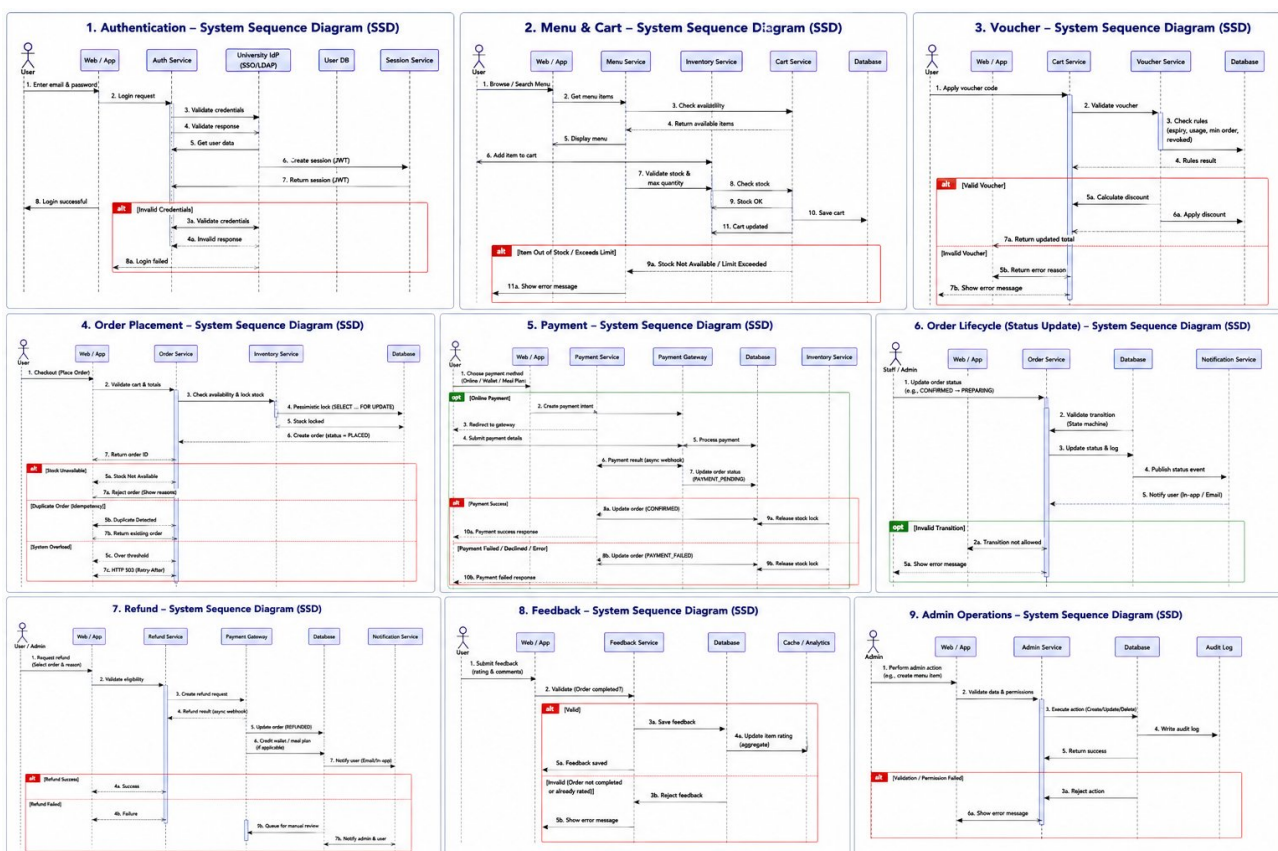
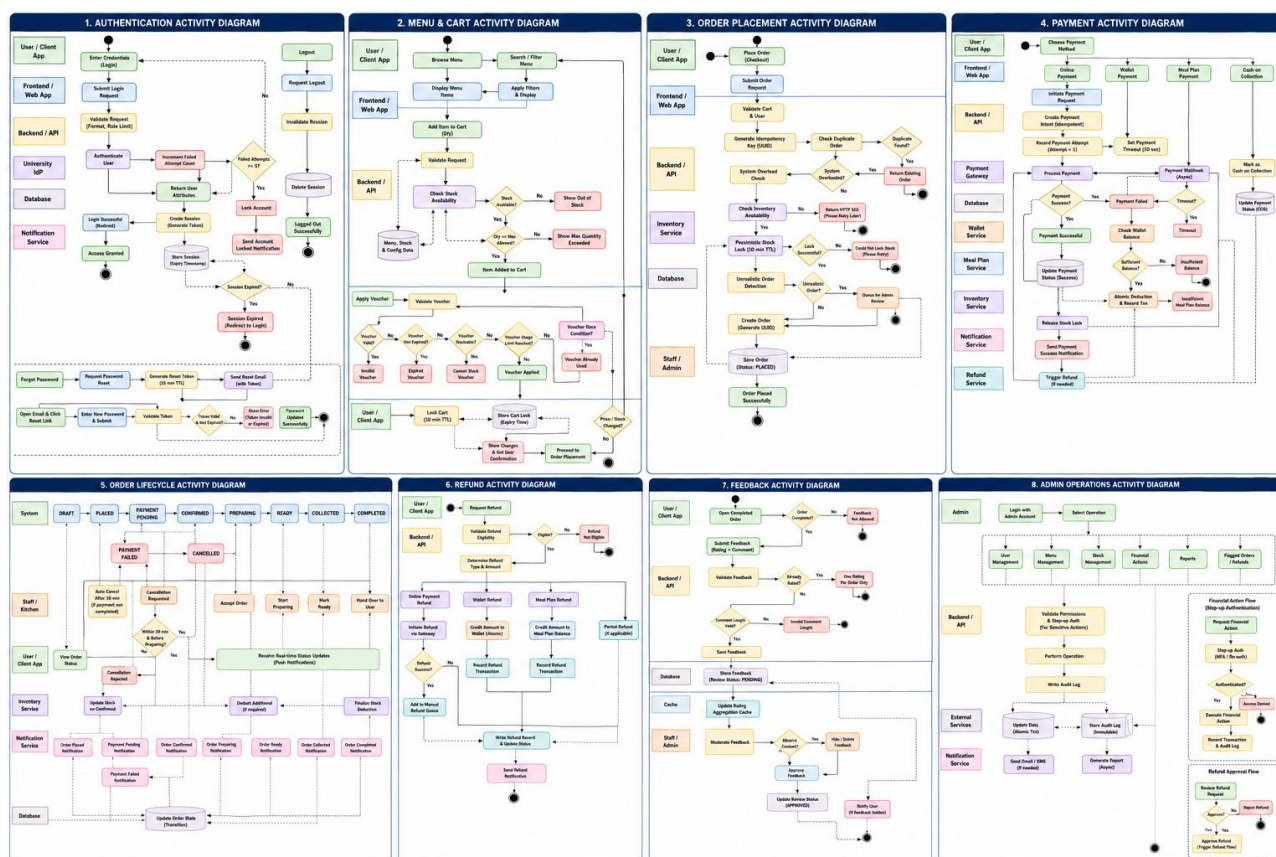


Figure 1: System Sequence Diagrams — Authentication, Menu & Cart, Voucher, Order Placement, Payment, Order Lifecycle, Refund, Feedback, and Admin Operations.

Activity Diagrams

Eight activity diagrams model the detailed control flow and decision logic for each functional area, showing swimlane responsibilities across the User/Client App, Frontend/Web App, Backend/API, and supporting services.



Part D: API Contracts & Information Hiding

Clear interface definitions for all system capabilities. Internal implementation details are explicitly hidden. Contracts enable independent front-end, back-end, and QA team development without knowledge of the internal stack.

POST /auth/login

Authenticate a user via institutional IdP. Returns JWT access and refresh tokens.

Request:

```
{ "email": "string (university email, required)",  
  "password": "string (min 8 chars, required)" }
```

Response (200 / 2xx):

```
{ "access_token": "string (JWT, TTL 1800s)",  
  "refresh_token": "string (JWT, TTL 604800s)",  
  "token_type": "Bearer",  
  "expires_in": 1800 }
```

Error Responses:

- 401 — Invalid credentials
- 403 — Account locked (includes Retry-After header)
- 403 — Account suspended
- 429 — Rate limit exceeded (20 req/min/IP)

Internal Implementation Hidden:

IdP URL/type, bcrypt cost factor, lockout counter storage, Redis key schema

POST /auth/logout

Invalidate all active sessions for the authenticated user across all devices.

Request:

```
Headers: Authorization: Bearer
```

Response (200 / 2xx):

```
HTTP 204 No Content
```

Error Responses:

- 401 — Token invalid or already expired

Internal Implementation Hidden:

Redis blacklist implementation, token invalidation mechanism

GET /menu

Retrieve all active menu items. Supports filtering by category and text search.

Request:

```
category: string (optional – Meals | Beverages | Snacks)  
q: string (optional – full-text search keyword)  
page: integer (default 1) | per_page: integer (default 20, max 100)
```

Response (200 / 2xx):

```
{ "items": [ { "id": "uuid", "name": "string",  
"price_egg": "number", "category": "string",  
"in_stock": "boolean", "avg_rating": "number | null",  
"max_order_qty": "integer" } ],  
"total": "integer", "page": "integer", "per_page": "integer" }
```

Error Responses:

- 400 — Invalid category value
- 429 — Rate limit

Internal Implementation Hidden:

DB query plan, CDN provider, cache TTL, stock table schema

POST /orders

Place an order from the locked cart. Idempotent within a 60-second window.

Request:

```
{ "cart_id": "uuid (required)",  
"idempotency_key": "string (required – client-generated UUID)" }
```

Response (200 / 2xx):

```
{ "order_id": "uuid (v4)", "status": "PLACED",  
"placed_at": "ISO 8601 UTC", "total_egg": "number",  
"stock_lock_expires_at": "ISO 8601 UTC" }
```

Error Responses:

- 200 — Duplicate: returns existing order (idempotency, within 60 s)
- 409 — One or more items unavailable (itemised reasons provided)
- 422 — Cart is empty or not locked
- 503 — System load threshold exceeded (Retry-After header provided)

Internal Implementation Hidden:

Idempotency key hashing algorithm, stock lock TTL configuration, queue depth monitoring

POST /orders/{order_id}/payment

Initiate payment for a PLACED order. Payment is processed asynchronously via gateway webhook.

Request:

```
{ "payment_method": "ONLINE | CASH | WALLET | MEAL_PLAN",  
"idempotency_key": "string (required – prevents double charge)" }
```

Response (200 / 2xx):

```
{ "payment_id": "uuid", "order_id": "uuid",  
"status": "PAYMENT_PENDING", "amount_egg": "number",  
"gateway_redirect_url": "string | null (ONLINE method only)" }
```

Error Responses:

- 402 — Insufficient wallet or Meal Plan balance
- 409 — Order not in PLACED status
- 422 — Invalid payment method
- 503 — Payment gateway unavailable (circuit breaker open)

Internal Implementation Hidden:

Gateway provider, API keys, webhook endpoint, circuit breaker state, idempotency key storage

Information Hiding Principles Applied

Principle	Implementation Detail
Database Schema Opacity	No SQL table names, column names, or join structures are ever returned in API responses.
Technology Stack Concealment	Framework names, ORM libraries, cache providers, and queue technologies are absent from all public contracts.
Internal Error Code Sanitisation	Stack traces and database error codes (e.g. ORA-00001) are never exposed to clients.
Gateway Abstraction	Payment gateway provider names, API keys, and webhook signing secrets are hidden from all consumers.
Algorithm Opacity	Password hashing algorithm and cost factor, token signing algorithm, and key rotation schedule are internal.
Configuration Abstraction	System thresholds (load limit, stock lock TTL, suspicious-order ceiling) are runtime config, not public contract.

Phase 3 — Test-Driven Implementation (TDI)

Methodology Overview — Test-Driven Prompting (TDP)

Phase 3 applies Test-Driven Prompting (TDP) — a structured three-step loop that governs every unit of AI-assisted implementation:

1. The Failing Test — A unit test is written first, establishing the precise mathematical/logical boundary the implementation must satisfy.
2. The Edge Case Cage — Padlocks are attached to each test: explicit boundary constraints and threshold guards that the AI must not relax.
3. Iteration — The AI is prompted repeatedly using the failing test + padlocks as the specification, until all assertions pass without modification.

Coverage target: All 24 edge cases from the SRS + all 7 hidden requirements (HR-01 to HR-07) receive at least one TDP cycle.

Module Ownership

Module	Member	FRs Owned	Edge Cases
Auth & Identity	Member 1	FR01–FR08, FR50–FR51	FR03, FR04, FR06, FR08
Menu & Cart	Member 2	FR09–FR19	FR14, FR15, FR16, FR17, FR19
Order & Payment	Member 3	FR20–FR32	FR22, FR23, FR24, FR25, FR28, FR29, FR30, FR31, FR32
Stock & Resilience	Member 4	FR33–FR41	FR37, FR38, FR40, FR41
Lifecycle & Reports	Member 5	FR42–FR56	FR44, FR45

Member 1 — Auth & Identity

TDP-M1-01 · FR03 — Account Lockout After 5 Failed Attempts

Step 1 · The Failing Test

```
class TestAccountLockout:
def test_account_locked_on_fifth_consecutive_failure(self, db_session):
# 4 failures – account must still be open
for attempt in range(4):
result = service.login(email, 'wrongPassword!')
assert result.locked is False
# 5th failure – account must lock NOW
result = service.login(email, 'wrongPassword!')
assert result.locked is True
```

```

assert result.http_status == 403
assert result.lock_duration_seconds == 900 # EXACT: 15 min = 900s

def test_lock_duration_is_exactly_900_seconds(self, db_session):
    with freeze_time(lock_start + timedelta(seconds=899)):
        result = service.login(email, 'correctPassword1!')
        assert result.locked is True # still active
    with freeze_time(lock_start + timedelta(seconds=900)):
        result = service.login(email, 'correctPassword1!')
        assert result.locked is False # released at exactly t+900s

```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Attempt count is exact	Lock triggers at attempt == 5, not attempt >= 4
P2	Lock duration is exact	900 seconds, not "approximately 15 minutes"
P3	Audit log is mandatory	Every lockout must write to audit_log before returning HTTP 403
P4	Counter resets to zero	Successful login resets to 0, not to current - 1
P5	Non-5th attempt returns 401, not 403	HTTP 401 for wrong password; HTTP 403 only at lockout

Step 3 · Iteration Prompt

CONSTRAINTS (Padlocks – inviolable):

- Lockout triggers on the 5th consecutive failed attempt, atomically.
- Lock duration is exactly 900 seconds stored as an integer.
- Before returning HTTP 403, write an AuditLog row with: event, actor_email, ip_address, timestamp.
- On any successful login, set failed_attempts = 0 in the same transaction.
- Attempts 1-4 must return HTTP 401. Only attempt 5 returns HTTP 403.

TDP-M1-02 · FR04 — Session Expiry After 30-Minute Inactivity

Step 1 · The Failing Test

```

class TestSessionExpiry:
    def test_token_rejected_after_30_minutes_inactivity(self, redis_client):
        with freeze_time(NOW + timedelta(seconds=1799)):
            result = validate_token(token, redis_client)
            assert result.valid is True # still alive at 1799s
        with freeze_time(NOW + timedelta(seconds=1800)):
            result = validate_token(token, redis_client)
            assert result.valid is False
            assert result.http_status == 401
            assert result.message == "Session expired. Please log in again."

    def test_activity_resets_inactivity_clock(self, redis_client):
        with freeze_time(NOW + timedelta(seconds=1000)):

```

```
touch_session(token, redis_client) # activity event
with freeze_time(NOW + timedelta(seconds=2799)):
    assert validate_token(token, redis_client).valid is True # 1000+1800 > 2799
with freeze_time(NOW + timedelta(seconds=2800)):
    assert validate_token(token, redis_client).valid is False
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Inactivity window is exact	1800s, not "about 30 minutes"
P2	Server-side invalidation is mandatory	Token must be removed from Redis — client-side expiry is insufficient
P3	Clock is inactivity-based, not issuance-based	Activity resets the countdown; the window is from last_active
P4	Error message is exact	Must read: "Session expired. Please log in again."

TDP-M1-03 · FR06 — Password Reset Link (15-Minute TTL, Single-Use)

Step 1 · The Failing Test

```
class TestPasswordReset:
    def test_reset_link_expires_after_15_minutes(self, db_session):
        with freeze_time(NOW + timedelta(seconds=899)):
            result = consume_reset_token(token, db=db_session)
            assert result.valid is True
        with freeze_time(NOW + timedelta(seconds=900)):
            result = consume_reset_token(token, db=db_session)
            assert result.valid is False
            assert result.error == "LINK_EXPIRED"

    def test_reset_link_is_single_use(self, db_session):
        first = consume_reset_token(token, new_password='NewPass1!', db=db_session)
        assert first.valid is True
        second = consume_reset_token(token, new_password='AnotherPass1!', db=db_session)
        assert second.valid is False
        assert second.error == "LINK_ALREADY_USED"
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	TTL is exactly 900 seconds	899s → valid; 900s → expired
P2	Single-use is enforced atomically	Token marked used=True in the same transaction as password update
P3	Expired token must not change password	No side effects on expiry — password hash unchanged

#	Padlock	Boundary
P4	Distinct error codes	LINK_EXPIRED ≠ LINK_ALREADY_USED — client can show different messages

TDP-M1-04 · FR08 — Reject Suspended / Expired Accounts

Step 1 · The Failing Test

```
@pytest.mark.parametrize('status,expected_message', [
    ('SUSPENDED', 'Your account has been suspended. Contact the registrar.'),
    ('EXPIRED', 'Your university account has expired. Contact IT services.'),
    ('NOT_FOUND', 'No account found for this email address.'),
])
def test_rejected_account_statuses(db_session, status, expected_message):
    result = AuthService(db_session).login(email, 'validPass1!')
    assert result.http_status == 403
    assert result.message == expected_message
    assert result.access_token is None # PADLOCK: no token ever issued
    assert result.refresh_token is None
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	No token on rejection	access_token and refresh_token are None for all rejected statuses
P2	Messages are status-specific	SUSPENDED ≠ EXPIRED ≠ NOT_FOUND messages
P3	HTTP status is 403, not 401	Suspended is authorization failure, not authentication failure

Member 2 — Menu & Cart

TDP-M2-01 · FR14–FR16 — Voucher Validation Edge Cases

Step 1 · The Failing Test

```
class TestVoucherValidation:
    def test_voucher_floors_total_at_zero_never_negative(self, db_session):
        cart = build_cart(total_egg=30.00)
        voucher = build_voucher(flat_discount_egg=100.00)
        result = apply_voucher(cart, voucher, db=db_session)
        assert result.final_total_egg == 0.00 # exact zero
        assert result.final_total_egg >= 0.00 # never negative
        assert result.discount_egg == 30.00 # capped at cart total
        assert result.excess_egg is None # no carryover field

    def test_concurrent_single_use_voucher_first_wins(self, db_session):
        results = run_concurrent([apply_voucher(120), apply_voucher(120)])
        assert len([r for r in results if r.http_status == 200]) == 1
```



```
assert len([r for r in results if r.http_status == 409]) == 1
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Final total floor is zero	final_total_egg >= 0 always
P2	Applied discount ≤ cart total	discount_egg is capped at cart_total, not at voucher_value
P3	Stacking rejected with specific code	VOUCHER_STACK_REJECTED code, not a generic 422
P4	Concurrent use: exactly 1 success	DB-level atomic check; second request gets HTTP 409
P5	All error messages are exact	No generic fallback message allowed

Step 3 · Iteration Prompt

CONSTRAINTS (Padlocks – cannot be relaxed):

1. final_total_egg = max(0, cart_total - voucher_discount). Never negative.
2. If the cart already has an applied voucher, return HTTP 422 immediately.
Error code: VOUCHER_STACK_REJECTED.
3. Single-use check must use DB-level atomic operation (SELECT FOR UPDATE).
4. Error messages for each rejection must match exact strings in the parametrized test.

TDP-M2-02 · FR17 — Cart Lock at Checkout

Step 1 · The Failing Test

```
class TestCartLock:
    def test_cart_is_read_only_after_checkout_initiated(self, db_session):
        cart = create_and_lock_cart(user_id='user-001', db=db_session)
        result = add_item_to_cart(cart.id, item_id='item-xyz', db=db_session)
        assert result.http_status == 409
        assert result.error == 'CART_LOCKED'

    def test_price_change_detected_on_lock(self, db_session):
        cart = build_cart_with_item('koshary', price_at_add=35.00)
        change_item_price('koshary', new_price=40.00, db=db_session)
        result = lock_cart_for_checkout(cart.id, db=db_session)
        assert result.warnings is not None
        # warnings[] contains old_price=35.00, new_price=40.00
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Locked cart rejects all mutations	HTTP 409 CART_LOCKED on any POST/PATCH to cart
P2	Price drift is detected and reported	warnings[] array with old_price and new_price fields
P3	Stock depletion blocks locking	HTTP 409 if any item reaches 0 stock before lock completes

TDP-M2-03 · FR19 — Max Order Quantity Per Item

Step 1 · The Failing Test

```
class TestMaxOrderQuantity:
    def test_quantity_at_cap_is_accepted(self, db_session):
        set_item_max_qty('grilled_chicken', max_qty=10, db=db_session)
        result = add_to_cart('grilled_chicken', quantity=10)
        assert result.http_status == 200

    def test_quantity_one_above_cap_is_rejected(self, db_session):
        result = add_to_cart('grilled_chicken', quantity=11)
        assert result.http_status == 422
        assert result.error == 'QUANTITY_EXCEEDS_CAP'
        assert result.max_allowed == 10

    def test_cap_is_per_item_not_per_cart(self, db_session):
        # 10 of item A + 10 of item B = 20 total, each within its own cap
        result = add_multiple_to_cart([('grilled_chicken', 10), ('koshary', 10)])
        assert result.http_status == 200
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Cap is per-item, not per-cart	10 of item A + 10 of item B = 20 total → allowed
P2	Response includes max_allowed	Client needs this to display correct UI message
P3	Cap enforced at add-to-cart AND checkout	Both points must validate; not just one

Member 3 — Order & Payment

TDP-M3-01 · FR22 — Pessimistic Stock Lock (Oversell Prevention)

Step 1 · The Failing Test

```
class TestStockLock:
    def test_last_item_cannot_be_oversold_under_concurrency(self, db_session):
        set_stock('koshary', quantity=1, db=db_session)
        results = run_concurrent([
            lambda: place_order(user='user-001', items=[('koshary',1)], db=db_session),
            lambda: place_order(user='user-002', items=[('koshary',1)], db=db_session),
        ])
        confirmed = [r for r in results if r.http_status == 200]
        rejected = [r for r in results if r.http_status == 409]
        assert len(confirmed) == 1
        assert len(rejected) == 1
        assert rejected[0].error == 'OVERSELL_PREVENTED'

    def test_lock_ttl_is_exactly_10_minutes(self, db_session):
        with freeze_time(NOW + timedelta(seconds=599)):
```

```
assert get_available_stock('koshary') == 0 # still locked
with freeze_time(NOW + timedelta(seconds=600)):
    assert get_available_stock('koshary') == 1 # released
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Exactly 1 winner under concurrency	Stock=1, 2 concurrent requests → 1 success, 1 rejection
P2	Lock TTL is exactly 600 seconds	Lock held at 599s; released at 600s
P3	Payment failure releases full lock	Available stock returns to pre-order level
P4	No partial release	If order has 3 items locked, all 3 are released atomically

Step 3 · Iteration Prompt

CONSTRAINTS:

- Use SELECT FOR UPDATE NOWAIT to acquire stock. If lock fails → HTTP 409 OVERSELL_PREVENTED.
- Stock lock TTL is exactly 600 seconds. Background job releases expired locks every 60 s.
- On PAYMENT_FAILED or PAYMENT_TIMEOUT, release ALL stock locks in a single atomic transaction.

TDP-M3-02 · FR23 — Duplicate Order Idempotency (60-Second Window)

Step 1 · The Failing Test

```
class TestOrderIdempotency:
    def test_duplicate_within_60s_returns_original_order(self, db_session):
        key = 'idem-key-abc-123'
        first = place_order(cart_id='cart-001', idempotency_key=key)
        second = place_order(cart_id='cart-001', idempotency_key=key)
        assert second.order_id == first.order_id
        assert db_session.query(Order).filter_by(idempotency_key=key).count() == 1

    def test_new_order_allowed_after_60_second_window(self, db_session):
        first = place_order(cart_id='cart-001', idempotency_key=key)
        with freeze_time(NOW + timedelta(seconds=61)):
            second = place_order(cart_id='cart-001', idempotency_key=key)
            assert second.order_id != first.order_id
            assert db_session.query(Order).filter_by(idempotency_key=key).count() == 2
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Window is exactly 60 seconds	At 59s → deduplicate; at 61s → new order
P2	Deduplication is key-based, not content-based	Same cart with different key → 2 orders
P3	Only 1 DB row per key within window	Not 2 rows returning same ID

TDP-M3-03 · FR28–FR30 — Payment Gateway Failure, Timeout & Double-Charge Prevention

Step 1 · The Failing Test

```
class TestPaymentResilience:
    def test_gateway_timeout_after_10_seconds(self, db_session):
        with freeze_time(NOW + timedelta(seconds=9)):
            assert get_order_status(order.id) == 'PAYMENT_PENDING'
        with freeze_time(NOW + timedelta(seconds=10)):
            run_timeout_checker(db=db_session)
            assert get_order_status(order.id) == 'PAYMENT_FAILED'

    def test_retry_max_3_attempts(self, db_session):
        third_attempt = retry_payment(order.id, db=db_session)
        assert third_attempt.http_status == 200
        fourth_attempt = retry_payment(order.id, db=db_session)
        assert fourth_attempt.http_status == 422
        assert fourth_attempt.error == 'MAX_RETRIES_EXCEEDED'

    def test_idempotency_key_prevents_double_charge(self, db_session):
        simulate_gateway_success(order.id, idempotency_key=key) # request 1
        simulate_gateway_success(order.id, idempotency_key=key) # retry
        assert db_session.query(PaymentRecord).filter_by(idempotency_key=key).count() == 1
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Timeout is exactly 10 seconds	At 9s → PAYMENT_PENDING; at 10s → PAYMENT_FAILED
P2	Stock release is atomic with status change	Cannot have PAYMENT_FAILED + still-locked stock
P3	Max retries is exactly 3	3rd → allowed; 4th → MAX_RETRIES_EXCEEDED
P4	Double-charge blocked by idempotency key	Same key → 1 PaymentRecord row in DB
P5	Meal Plan response includes all three money fields	current_balance, required, and shortfall all present

TDP-M3-04 · FR25 — System Load-Shedding (HTTP 503)

Step 1 · The Failing Test

```
class TestLoadShedding:
    def test_503_returned_when_concurrent_orders_at_limit(self, db_session):
        set_system_config('max_concurrent_orders', 150)
        seed_active_orders(count=150)
```

```

result = place_order(user='user-new', cart_id='cart-xyz')
assert result.http_status == 503
assert int(result.headers['Retry-After']) == 30
assert result.message == 'Service temporarily busy. Please try again shortly.'

def test_order_accepted_when_one_below_limit(self, db_session):
    seed_active_orders(count=149)
    result = place_order(user='user-new', cart_id='cart-xyz')
    assert result.http_status in (200, 201)

```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Threshold is configurable and respected exactly	150 active → 503; 149 active → 200
P2	Retry-After header is required	Must be present and set to 30
P3	Message is exact, no stack trace	"Service temporarily busy. Please try again shortly."

Member 4 — Stock & Resilience

TDP-M4-01 · FR37–FR38 — User Cancellation Window (2 Minutes)

Step 1 · The Failing Test

```

class TestCancellationWindow:
    def test_cancellation_allowed_at_119_seconds(self, db_session):
        with freeze_time(NOW + timedelta(seconds=119)):
            result = cancel_order(order.id, user_role='student')
            assert result.http_status == 200
            assert result.status == 'CANCELLED'

    def test_cancellation_rejected_at_120_seconds(self, db_session):
        with freeze_time(NOW + timedelta(seconds=120)):
            result = cancel_order(order.id, user_role='student')
            assert result.http_status == 403
            assert result.error == 'CANCELLATION_WINDOW_EXPIRED'

    def test_staff_cancel_without_reason_code_rejected(self, db_session):
        result = cancel_order(order.id, user_role='staff', reason_code=None)
        assert result.http_status == 422
        assert result.error == 'REASON_CODE_REQUIRED'

```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Window boundary is exactly 120 seconds	119s → allowed; 120s → rejected

#	Padlock	Boundary
P2	Status also blocks cancellation independently	PREPARING blocks user regardless of elapsed time
P3	Staff cancellation requires reason_code	reason_code=None → HTTP 422
P4	Error messages are status-specific	Window expiry ≠ status block → different messages

Step 3 · Iteration Prompt

CONSTRAINTS:

- For student role: check BOTH (1) time window AND (2) status independently.
- For staff/admin role: only COMPLETED and CANCELLED are blocked; all others cancellable with reason_code.
- Valid reason_codes: CUSTOMER_REQUEST, OUT_OF_STOCK, STAFF_ERROR, SYSTEM_ERROR, SUSPICIOUS_ORDER.
- Write to audit_log before returning any response. No silent cancellations.

TDP-M4-02 · FR40 — Auto-Cancel Abandoned Checkout (10-Minute TTL)

Step 1 · The Failing Test

```
class TestAbandonedCheckoutAutoCancellation:
    def test_auto_cancelled_at_exactly_10_minutes(self, db_session):
        with freeze_time(NOW + timedelta(seconds=599)):
            run_cleanup_job()
            assert get_order_status(order.id) == 'PAYMENT_PENDING'
        with freeze_time(NOW + timedelta(seconds=600)):
            run_cleanup_job()
            assert get_order_status(order.id) == 'CANCELLED'

    def test_student_notified_on_auto_cancel(self, mock_notification):
        with freeze_time(NOW + timedelta(seconds=600)):
            run_cleanup_job()
            mock_notification.assert_called_once_with(
                user_id='user-001',
                message='Your pending order was automatically cancelled due to payment timeout.'
            )
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	TTL is exactly 600 seconds	599s → still PAYMENT_PENDING; 600s → CANCELLED
P2	Stock release is part of auto-cancel transaction	Cannot cancel without releasing locks
P3	Notification is mandatory	No silent auto-cancellations

TDP-M4-03 · FR41 — Stock Inconsistency Post-Confirmation

Step 1 · The Failing Test

```
class TestStockInconsistency:
    def test_order_held_when_stock_inconsistency_detected(self, db_session):
        order = create_confirmed_order(items=[('koshary', 3)])
        force_stock_inconsistency('koshary', available=1)
        run_stock_consistency_checker()
        assert db_session.query(Order).get(order.id).status == 'HELD'

    def test_admin_notified_on_stock_inconsistency(self, mock_admin_alert):
        mock_admin_alert.assert_called_once()
        alert = mock_admin_alert.call_args[0][0]
        assert alert['order_id'] == order.id
        assert alert['type'] == 'STOCK_INCONSISTENCY'
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Inconsistency detected post-confirmation → HELD	Order status set to HELD, not CANCELLED
P2	Admin queue entry created	Not just an internal log — visible in admin review queue

Member 5 — Lifecycle & Reports

TDP-M5-01 · FR42–FR43 — Refund Initiation & Wallet Idempotency

Step 1 · The Failing Test

```
class TestRefunds:
    def test_wallet_refund_is_atomic_and_immediate(self, db_session):
        set_wallet_balance('user-001', balance=100.00)
        order = create_confirmed_wallet_order(user='user-001', total=60.00)
        cancel_order(order.id, reason_code='CUSTOMER_REQUEST')
        assert get_wallet_balance('user-001') == 100.00 # fully restored

    def test_wallet_refund_is_idempotent(self, db_session):
        process_refund(order.id, refund_reference='REF-001')
        process_refund(order.id, refund_reference='REF-001') # retry
        assert get_wallet_balance('user-001') == 160.00 # 100 + 60 once only

    def test_failed_gateway_refund_added_to_manual_queue(self, db_session):
        simulate_gateway_refund_failure(order.id)
        queue_item = db_session.query(ManualRefundQueue).filter_by(order_id=order.id).first()
        assert queue_item is not None
        assert queue_item.status == 'PENDING_MANUAL'
        assert queue_item.amount_egp == 120.00
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Wallet refund in same transaction as cancellation	No window where order is CANCELLED but wallet not credited
P2	Refund idempotency prevents double-credit	Same refund_reference → credited exactly once
P3	Gateway failure creates queue entry — no silent drop	ManualRefundQueue row must exist

Step 3 · Iteration Prompt

CONSTRAINTS:

- Wallet refunds must execute inside the same DB transaction as the order status update to CANCELLED.
- Idempotency: check if a refund record with this refund_reference already exists before any DB write.
- On gateway refund API failure: write to manual_refund_queue with status=PENDING_MANUAL.
- audit_log must record every refund attempt (success, failure, and duplicate-skip).

TDP-M5-02 · FR45 — Partial Refund on Partial Fulfilment

Step 1 · The Failing Test

```
class TestPartialRefund:
def test_partial_refund_amount_matches_unfulfilled_items_only(self, db_session):
# Order: koshary 35 EGP (FULFILLED) + grilled_chicken 65 EGP (NOT_FULFILLED)
# + juice 20 EGP (FULFILLED)
result = process_partial_refund(order.id, unfulfilled_items=['grilled_chicken'])
assert result.refund_amount_egp == 65.00 # only the unfulfilled item
assert result.refund_amount_egp != 120.00 # not full order total
assert result.refund_amount_egp != 55.00 # not sum of fulfilled items

def test_partial_refund_requires_staff_authorization(self, db_session):
result = process_partial_refund(order.id, unfulfilled_items=['grilled_chicken'],
actor_role='student')
assert result.http_status == 403
assert result.error == 'INSUFFICIENT_ROLE'
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Refund is sum of unfulfilled items only	Not total order; not fulfilled items
P2	Only staff/admin can trigger partial refund	Student actor → HTTP 403
P3	Refund recorded in audit_log	Append-only; no edits permitted

TDP-M5-03 · FR47 — Feedback Submission (Post-COMPLETED Only)

Step 1 · The Failing Test

```
class TestFeedback:
    def test_rating_accepted_only_for_completed_order(self, db_session):
        completed = create_order_in_status('COMPLETED')
        assert submit_rating(completed.id, stars=4).http_status == 200

    @pytest.mark.parametrize('status',
                              ['PLACED', 'CONFIRMED', 'PREPARING', 'READY', 'CANCELLED'])
    def test_rating_blocked_for_non_completed_statuses(self, status):
        result = submit_rating(order.id, stars=5)
        assert result.http_status == 403

    def test_rating_cannot_be_edited_after_submission(self, db_session):
        submit_rating(order.id, stars=4)
        second = submit_rating(order.id, stars=2)
        assert second.http_status == 409
        assert second.error == 'RATING_ALREADY_SUBMITTED'
        assert db_session.query(Rating).filter_by(order_id=order.id).first().stars == 4

    def test_star_rating_must_be_between_1_and_5(self, db_session):
        assert submit_rating(order.id, stars=0).http_status == 422
        assert submit_rating(order.id, stars=6).http_status == 422
        assert submit_rating(order.id, stars=1).http_status == 200
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Rating allowed for COMPLETED only	All 5 other statuses → 403
P2	No editing after submission	Second submit → 409, original rating unchanged
P3	Stars range: 1–5 inclusive	0 and 6 → 422; 1 and 5 → 200

Hidden Requirements — TDP Cycles

TDP-HR-01 · HR-02 — Meal Plan Activation Lag

Step 1 · The Failing Test

```
class TestMealPlanActivationLag:
    def test_pending_activation_is_distinct_from_insufficient_balance(self, db_session):
        set_meal_plan_status('user-new', status='PENDING_ACTIVATION')
        result_pending = pay_with_meal_plan(order_total=50.00, user='user-new')
        assert result_pending.error == 'MEAL_PLAN_NOT_YET_ACTIVATED'
        assert result_pending.message == (
            'Your Meal Plan is pending activation. '
            'Please contact the student services office.'
        )
```

```

set_meal_plan_status('user-new', status='ACTIVE', balance=0.00)
result_zero = pay_with_meal_plan(order_total=50.00, user='user-new')
assert result_zero.error == 'INSUFFICIENT_MEAL_PLAN_BALANCE'
assert result_zero.error != 'MEAL_PLAN_NOT_YET_ACTIVATED'

```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Two distinct error codes required	MEAL_PLAN_NOT_YET_ACTIVATED ≠ INSUFFICIENT_MEAL_PLAN_BALANCE
P2	PENDING_ACTIVATION checked before balance	Activation status check is the first gate

TDP-HR-04 · HR-04 — Staff Order Assignment Idempotency

Step 1 · The Failing Test

```

class TestStaffOrderIdempotency:
def test_concurrent_staff_transitions_are_idempotent(self, db_session):
order = create_order_in_status('CONFIRMED')
results = run_concurrent([
lambda: advance_order_status(order.id, actor='staff-A', to_status='PREPARING'),
lambda: advance_order_status(order.id, actor='staff-B', to_status='PREPARING'),
])
assert len([r for r in results if r.http_status == 200]) == 1
assert len([r for r in results if r.http_status == 409]) == 1
assert db_session.query(Order).get(order.id).status == 'PREPARING'
assert db_session.query(OrderTransitionLog).filter_by(order_id=order.id).count() == 1

```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Exactly 1 successful transition under concurrency	DB-level lock prevents double-advance
P2	Exactly 1 row in OrderTransitionLog	No duplicate log entries for same transition
P3	HTTP 409 for the loser	ORDER_ALREADY_ADVANCED error code

TDP-HR-07 · HR-07 — Cross-Device Cart Invalidation

Step 1 · The Failing Test

```

class TestCrossDeviceCart:
def test_stale_cart_invalidated_on_second_device(self, db_session):
cart = create_cart(user_id='user-001')
# Device A locks and proceeds to checkout
lock_cart_for_checkout(cart.id, session_id='session-device-A')
# Device B tries to checkout the same cart

```

```
result = lock_cart_for_checkout(cart.id, session_id='session-device-B')
assert result.http_status == 409
assert result.error == 'CART_INVALIDATED'
assert result.message == (
    'Your cart was modified on another device. '
    'Please refresh to see the latest state.'
)
```

Step 2 · The Edge Case Cage (Padlocks)

#	Padlock	Boundary
P1	Cart locked by session A blocks session B	Second lock attempt → HTTP 409
P2	Error code is CART_INVALIDATED, not CART_LOCKED	Cross-device scenario has its own error code

Phase 4 — Testing & Quality Assurance Report

1. Introduction & Testing Strategy

CampusBite is a campus cafeteria management system that handles authentication, menu browsing, cart management, order placement, payment processing, and refunds. Given the system's focus on transactional integrity, concurrency control, and real-time stock management, a comprehensive multi-layer testing strategy is essential.

This report presents the full testing architecture for Phase 4, covering unit tests, integration tests, end-to-end tests, performance tests, security tests, and CI/CD pipeline configuration. All tests validate both functional requirements (FR) and non-functional requirements (NFR) defined in the SRS.

2. Testing Pyramid Distribution

Layer	Coverage	Tools	Focus
Unit Tests	70%	Pytest, Vitest, React Testing Library	Individual functions & components
Integration Tests	20%	Pytest, Test DB, Redis	API + DB, transactions, concurrency
E2E / System Tests	10%	Playwright + POM	Full user flows, Gherkin scenarios

Higher-level tests (Performance, Security) complement the pyramid and address the system's NFRs around concurrency, response time, and OWASP compliance.

3. Unit Testing (70%)

Unit tests validate individual functions and components in isolation, using mocks and stubs to eliminate external dependencies. The goal is fast feedback and high branch coverage on business logic.

3.1 Backend Unit Tests — Authentication

Install dependencies:

```
pip install pytest pytest-asyncio httpx pytest-cov pytest-mock
```

Folder structure:

```
backend/tests/unit/auth/  
test_login.py  
test_jwt.py  
test_lockout.py  
test_roles.py
```

Coverage areas: login validation, JWT generation/expiration, password hashing verification, account logout after 5 consecutive failed attempts (FR03), role-based access validation.

Example — Account Lockout Test:

```
def test_user_locked_after_5_failed_attempts():  
    for _ in range(5):  
        login('wrong_password')  
    response = login('wrong_password')  
    assert response.status_code == 423
```

3.2 Frontend Unit Tests

Install dependencies:

```
npm install -D vitest @testing-library/react @testing-library/jest-dom
```

Example — Out-of-Stock Badge (FR11):

```
test('shows out of stock badge', () => {  
    render()  
    expect(screen.getByText(/out of stock/i)).toBeInTheDocument()  
})
```

FR11 requires that out-of-stock items be non-selectable. This test verifies the component renders the correct badge and disables the add-to-cart button.

4. Integration Testing (20%)

Integration tests validate the interaction between components: API endpoints, the database, Redis cache, and the payment service. This layer is critical due to SRS requirements around concurrent orders, atomic transactions, and idempotency.

Folder structure:

```
backend/tests/integration/  
test_order_flow.py  
test_payment_flow.py  
test_stock_locking.py  
test_wallet_atomicsity.py  
test_refunds.py
```

4.1 Stock Locking — FR22

```
async def test_last_item_can_only_be_bought_once():  
    responses = await asyncio.gather(  
        place_order(user1),  
        place_order(user2)
```

```
)
success = [r for r in responses if r.status_code == 200]
assert len(success) == 1
```

4.2 Payment Idempotency — FR30

```
def test_payment_idempotency():
    first = pay(idempotency_key='abc')
    second = pay(idempotency_key='abc')
    assert first.transaction_id == second.transaction_id
```

4.3 Wallet Atomicity — FR32

```
def test_wallet_atomicity_on_order_failure():
    balance_before = get_wallet_balance(user1)
    force_order_failure_after_deduction(user1)
    balance_after = get_wallet_balance(user1)
    assert balance_before == balance_after
```

5. End-to-End Testing (10%) — Playwright + POM

E2E tests simulate real user journeys in a browser. Playwright is used with the Page Object Model (POM) pattern to keep tests maintainable. Gherkin scenarios from the SRS are converted directly into executable Playwright scripts.

Setup:

```
npm init playwright@latest
```

Folder structure:

```
frontend/playwright/
e2e/
auth.spec.ts order.spec.ts
payment.spec.ts refund.spec.ts
pages/
LoginPage.ts CartPage.ts
CheckoutPage.ts OrdersPage.ts
```

Page Object Model — LoginPage

```
export class LoginPage {
    constructor(private page: Page) {}
    async goto() { await this.page.goto('/login') }
    async login(email: string, password: string) {
        await this.page.fill('#email', email)
        await this.page.fill('#password', password)
        await this.page.click('button[type=submit]')
    }
}
```

Gherkin → Playwright Conversion

Gherkin Scenario

```
Scenario: Successful Order
Given user is logged in
When user adds item to cart
And completes payment
Then order becomes CONFIRMED
```

Converted Playwright Test

```
test('user places successful order', async ({ page }) => {
  const login = new LoginPage(page)
  await login.goto()
  await login.login('student@test.com', '123456')
  await page.click('text=Burger')
  await page.click('text=Checkout')
  await expect(page.locator('text=Order Confirmed')).toBeVisible()
})
```

6. Performance Testing

Performance tests address NFRs: handling 200 concurrent users, processing 500 stock locks per second, and maintaining response times under 2 seconds during peak hours.

Tool: k6 | **Folder:** backend/tests/performance/

```
import http from 'k6/http'
import { sleep } from 'k6'

export let options = { vus: 200, duration: '30s' }

export default function () {
  http.post('http://localhost:8000/orders')
  sleep(1)
}
```

NFR	Threshold	Test File
Concurrent Users	200 simultaneous users	order-load.js
Stock Lock Throughput	500 locks/sec	payment-stress.js
Response Time	< 2 seconds (p95)	search-load.js

7. Security Testing

Security tests verify OWASP compliance, JWT handling, rate limiting, and role-based access control. These map directly to the system's NFRs.

Folder: `backend/tests/security/` | **Files:** `test_sql_injection.py`, `test_rate_limit.py`, `test_jwt_expiry.py`, `test_rbac.py`

SQL Injection Prevention

```
def test_sql_injection_rejected():
    response = client.post('/login', json={
        'email': "' OR 1=1 --",
        'password': 'x'
    })
    assert response.status_code == 401
```

Rate Limiting (NFR19)

```
def test_rate_limit_enforced():
    for _ in range(100):
        client.get('/menu')
    response = client.get('/menu')
    assert response.status_code == 429
```

8. CI/CD Pipeline

A GitHub Actions pipeline runs all test layers automatically on every push and pull request, ensuring no regression reaches production.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_DB: cafeteria, POSTGRES_PASSWORD: password }
    steps:
      - uses: actions/checkout@v4
      - name: Backend Tests
        run: |
          pip install -r requirements.txt
          pytest --cov
      - name: Frontend Tests
        run: |
          cd frontend/menu-cart && npm install && npm run test
      - name: Playwright E2E
        run: |
          cd frontend/menu-cart && npx playwright test
```


9. Validation vs. Verification

Verification — "Are we building the system right?"

Verification checks that the implementation conforms to the specification. It answers: does the code do what the SRS says it should do?

- Unit tests verify that authentication logic matches the SRS rules
- Integration tests verify that stock is correctly deducted on order placement
- Integration tests verify that payment is confirmed before order status changes
- E2E tests verify that the full order flow completes as specified

Validation — "Are we building the right system?"

Validation checks that the system solves the actual real-world problem for its users.

- Load tests validate that the system handles the lunch peak (200 concurrent users)
- UX testing validates that students can place orders easily on mobile devices
- Security tests validate that the system resists real-world attack vectors (OWASP)
- Overselling prevention tests validate that no student receives an unfillable order

Test Type	Category	Question Answered
Unit Tests	Verification	Does login logic match the SRS?
Integration Tests	Verification	Does stock deduction work atomically?
E2E Tests	Verification + Validation	Does the user journey complete end-to-end?
Load Tests	Validation	Does the system survive lunch peak?
Security Tests	Validation	Is the system safe for real users?

10. Requirements Coverage Matrix

The matrix below maps critical SRS requirements to their corresponding test types, ensuring full traceability between requirements and test cases.

Requirement	Description	Test Type	File
FR11	Out-of-stock items non-selectable	Unit	test_menu_item.tsx
FR22	Stock lock — prevent overselling	Integration + Load	test_stock_locking.py
FR23	Idempotency on duplicate requests	Integration	test_payment_flow.py

Requirement	Description	Test Type	File
FR25	Overload protection	Performance	order-load.js
FR28	Payment failure handling	Integration	test_payment_flow.py
FR30	Double charge prevention	Integration	test_payment_flow.py
FR32	Wallet atomicity	Integration	test_wallet_atomicity.py
FR40	Auto-cancel on timeout	Integration	test_order_flow.py
NFR19	Rate limiting	Security	test_rate_limit.py
NFR22	ACID transactions	Integration	test_wallet_atomicity.py